

Shrub Hub

DESIGN RATIONALE

Overview

The Design Rationale is intended to provide designers and maker information about the design process and design decisions behind the development of the Shrub Hub, a cheap, easy to assemble switch access hub.



Shrub Hub

DESIGN RATIONALE

Contents

Overview	1
Introduction	4
User Needs.....	5
Design Input	5
Requirements.....	5
Goals	5
Requirements.....	5
Constraints	5
Research.....	6
Commercially Available Options	6
DIY / Maker-Friendly Options	7
Prototyping	8
Initial Prototype	8
Physical Component / Enclosure	8
Electrical Components	9
Code Structure / Function.....	9
Prototype Decisions	10
Second Prototype.....	10
Physical Components/Enclosure.....	10
Electrical Components	11
Code Structure/Function	14
Testing.....	19
Test Methods	19
Test Switches.....	19
Test Results	19
User Testing	19
Feedback Questions.....	19
Feedback Summary.....	21
Design Output	23
Naming.....	23

DESIGN RATIONALE

Detailed Design	24
Overview	24
Electrical Components	29
Code Structure / Function.....	29
Validation and Verification Plan.....	33
Verification.....	Error! Bookmark not defined.
Switch Adapter.....	Error! Bookmark not defined.
Software.....	Error! Bookmark not defined.
Validation.....	44
Opportunities for Improvement	50
Functionality	52
Electrical Components	52
Code Structure / Function.....	52

Shrub Hub

DESIGN RATIONALE



Introduction

Many people experience difficulty using standard input devices such as keyboards, mice, or touchscreens due to physical, cognitive, or sensory limitations. Assistive switches provide an alternative way to interact with technology by allowing users to control digital devices through simple, reliable actions such as pressing a button, activating a sensor, or triggering a switch with minimal movement.

To be useful, assistive switches must connect to computers, tablets, and other digital systems in a way that the device can recognize as input. While commercial switch interfaces exist to provide this digital access, they are often expensive, limited in customization, or difficult to obtain, creating a barrier for many users, families, schools, and support organizations.

This project presents a low-cost, easy-to-build digital switch interface designed to increase access to assistive switch technology. The device allows users to connect one or more assistive switches to common digital devices, enabling interaction with software, communication tools, and other technology. By focusing on affordability, simplicity, and accessibility, this design aims to reduce barriers to digital access and support a wider range of users in engaging with technology on their own terms.

The Shrub Hub is a USB Assistive Switch Interface based on the [Adafruit TRRS Trinkey](#) hardware, a USB Key with a 3.5 mm jack and a microcontroller that was created based on a request from [ATMakers](#), and the [Adafruit Learning Guide](#) by Chris Young and Bill Binko.

ID	Need	Interest-Holder
UN01	I need to be able to control my computer with minimal switches	Primary User
UN02	A device that doesn't require a soldering iron or PCB is easier to build	Maker
UN03	I want a device that is easy to set up	Secondary User
UN04	I want a device that I can customize to my needs	Primary User
UN05	An inexpensive switch interface	Primary User, Secondary User

Design Input

Requirements

The goals and requirements outlined here can be used to assess if a device would meet the needs of a user and determine when a design is sufficient for release.

Goals

G01	Create a low-cost switch interface
G02	Create an easy to build switch interface
G03	Compatible with a computer without installing software
G04	Create a portable switch interface

Requirements

F01	Device must accept at least two 3.5 mm switch inputs
F02	Device must output HID keystrokes to the connected device

Evaluation Criteria

1. Durability
2. Reliability
3. Minimize cost
4. Minimize print time
5. Minimize material usage
6. Minimize number of individual 3D printed parts

Constraints

C01	Device cannot use a custom PCB
C02	Device cannot require soldering

Shrub Hub

DESIGN RATIONALE

Research

Research was conducted as part of the update for the Switch Interface 101 resource.

Commercially Available Options

Options that can be purchased but not made by a maker.

Swiftly

Title / Name of device	Swiftly
Link	https://www.orin.com/access/swifty/
Author	Origin Instruments
License	N/A
Cost	\$99 USD



Figure 1: Swiftly Switch Interface

The Swiftly is a digital switch interface that can connect up to two 3.5 mm switches, or up to three wireless 'Beam' switches. It can be used to send keystrokes to a connected device for switch scanning or computer control.

Requirements Met	Requirements Unmet
F01, F02	G01

Useful Design Features

Using three switches instead of two opens more options for computer control.

DESIGN RATIONALE

DIY / Maker-Friendly Options

Options that can be made by a maker.

Rocket Switch

Title	Rocket Switch
Link	https://www.makersmakingchange.com/s/product/rocket-switch-interface/01tJR000000694SYAQ
Author	Neil Squire Society
License	Attribution-ShareAlike4.0 International
Cost	\$50 CAD
Test Build (Y/N)	Y
Add to Library (Y/N)	Y

**Figure 2: Rocket Switch Interface**

The Rocket Switch is a USB switch interface that accepts up to two 3.5 mm switches to send keystrokes to the connected device.

Requirements Met	Requirements Unmet
G01, F01, F02	C01, C02, G02

Useful Design Features

The modes feature of the Rocket Switch was taken and added to the Shrub Hub.

Shrub Hub

DESIGN RATIONALE

Prototyping

Initial Prototype

The initial version as published on the Adafruit site.

Physical Component / Enclosure

The enclosure used on the device was provided by Adafruit. The version used was the FDM printed version, although they also provide an SLA compatible version.

The case can be found at the following link: [The Case](#).

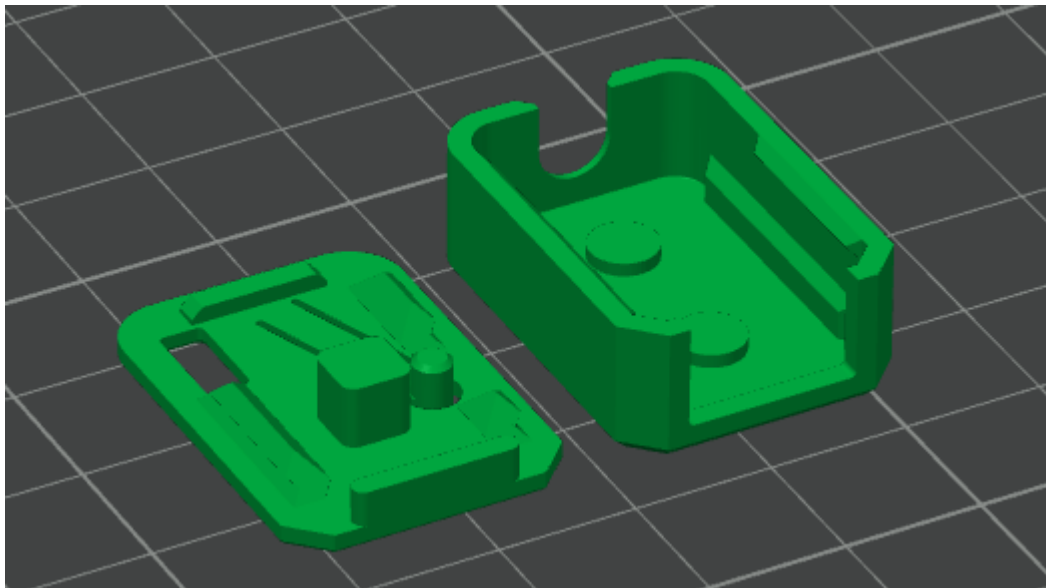
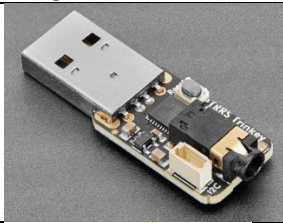
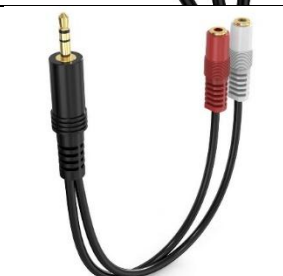


Figure 3: Adafruit TRRS Trinkey Enclosure

DESIGN RATIONALE

Electrical Components

Part	Image	Price	Link
Adafruit TRRS Trinkey		\$9.95 USD	Link
Microphone Splitter Cable		\$10.99 CAD	Link
Stereo to Mono Splitter		\$9.99 CAD	Link

Code Structure / Function

Code used for the initial version was the supplied code in the project repository called [TRRS Switch Control](#) on GitHub.

The code scans the connected buttons and determines which output to send depending on the combination of switches pressed. The software recognizes the 3 individual switch presses and has an additional 2 outputs from chorded inputs, with another 2 chords recognized but not implemented.

DESIGN RATIONALE

Prototype Decisions

Prototype	Decision (Abandon, Modify, Proceed)	Justification
Initial Prototype	Modify	Missing some functionality.

Second Prototype

The second prototype was developed based on the feedback received from the user testing. It involves changes to both the enclosure and the firmware.

Physical Components/Enclosure

An enclosure was developed to replace the one supplied by Adafruit for the TRRS Trinkey. This enclosure kept the same basic shape, with the snap-on lid with a flexible arm for the reset button, but several key changes were made.

The final enclosure has the lever arm, a light pipe for the NeoPixel LED, a USB-A connector, a TRRS plug, grippable spots on either side, and clip spots for attaching the TRRS breakout.

The TRRS breakout enclosure covers the TRRS terminal splitter and the cables. It prevents force on the cables from transferring to the screw terminals and makes a mechanical connection to the microcontroller that is sturdier than just the TRRS connection.

A new enclosure was developed that has the name of the device on it that includes a light pipe and covers the unused i2c port.

After user testing, a number of issues were raised about this enclosure. The testers found that:

- It could be difficult to unplug the device
- Light pipe was more 'tactile' than the reset button, leading to people pressing the light pipe
- Connection between Switch Adapter and the main device seemed weak
- Cable ties could be poorly trimmed by a maker and remain sharp

To resolve these issues, the enclosure was modified to raise the arm of the reset button beyond the plane of the lid so that it was more tactile, and a grip was added on either side of the enclosure to make it easier to remove. Two detents were also added to allow the Switch Adapter to attach better.

The Switch Adapter was redesigned to add two snap fit arms that attach to the enclosure, a set of internal deformable arms that secure the cables and replaced the cable tie with snap fit arms to hold the enclosure together.

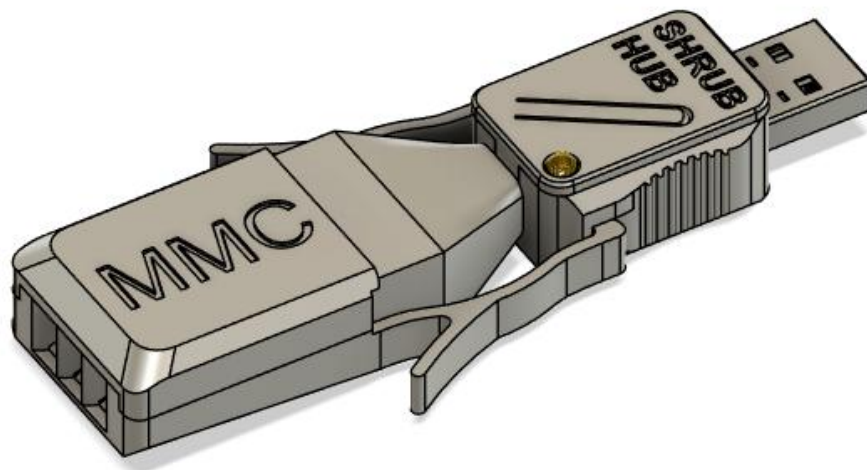


Figure 4: Shrub Hub and Switch Adapter Enclosure

Electrical Components

A piece of feedback received from the tester was that they would like to use the Shrub Hub with a single switch. The version that was given to the tester used the ring 2 as the common ground because that was the wiring used in the original Adafruit code, but plugging in a mono cable switch to this would short ring 1 and sleeve to ring 2, reading as switches 2 and 3 constantly being pressed down.

Version	L Terminal	R Terminal	V Terminal	Ground Terminal
Original	Switch 1	Switch 2	Ground	Switch 3
First Single Switch	Ground	Switch 1	Switch 2	Switch 3
Second Single Switch	Ground	Switch 2	Switch 1	Switch 3

To make this one switch version possible, the common ground was switched to the tip of the connector. This required the rewiring of the Switch Adapter to move the common ground, as well as some changes to the software. The ring 2 pin was changed to an input, while the tip was changed to an output. The switch detection function was modified to remove the tip, add ring 2, and change ring 1 to switch 1, ring 2 to switch 2, and sleeve to switch 3. Finally, since pressing a mono switch with this wiring would press all three switches, the chording needs to be added to handle this.

The original code recognized switch 6 (switch 1 + 3) and switch 7 (all three switches) but since they were not included in the keystroke matrix, the code does not send switch 6 or switch 7 to the keystroke compiler. Since the mono cable switch detects as switch 7, an else statement was added to make both switch 6 and 7 use the switch 1 entry in the keystroke matrix. It is possible to also add entries for switch 6 and 7 to the keystroke matrix, but due to the memory issues that the microcontroller faces, it was decided to test the one switch version before dealing with the memory management.

DESIGN RATIONALE

Once these changes were made, the device was tested with the switch plugged directly into the microcontroller, and it behaved as expected. The device was tested again using the Switch Adapter to ensure that it still functioned as before, and no issues were detected.

The one thing that stood out was the different behaviour when using a switch with a TRS jack instead of a mono jack. When using the TRS jack, pressing the switch only connects the ring 2 and sleeve to ground instead of all three, which is read by the device as switch 5, which is already programmed into the device. This does not throw any errors, but a user who does not know the difference between a mono cable and TRS cable and is expecting switch 1 might be confused as to why they are getting switch 5, especially if switch 5 is a 'non character', like the escape key or function keys.

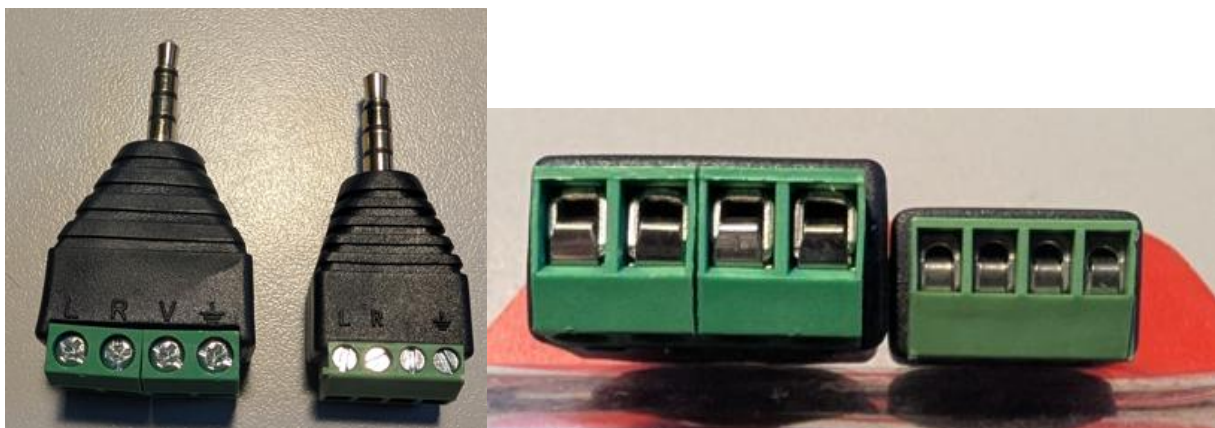
To resolve this issue, rings 1 and 2 were swapped, with ring 1 becoming switch 2 and ring 2 becoming switch 1. This causes a TRS switch to read as switch 6 when pressed, which behaves the same as switch 1, resolving the issue.

Alternate Parts Sourcing

When reviewing the parts in the Bill of Materials, we saw an opportunity to source the parts for the Switch Adapter through Digikey to simplify the ordering process and protect against the unreliability of Amazon sellers.

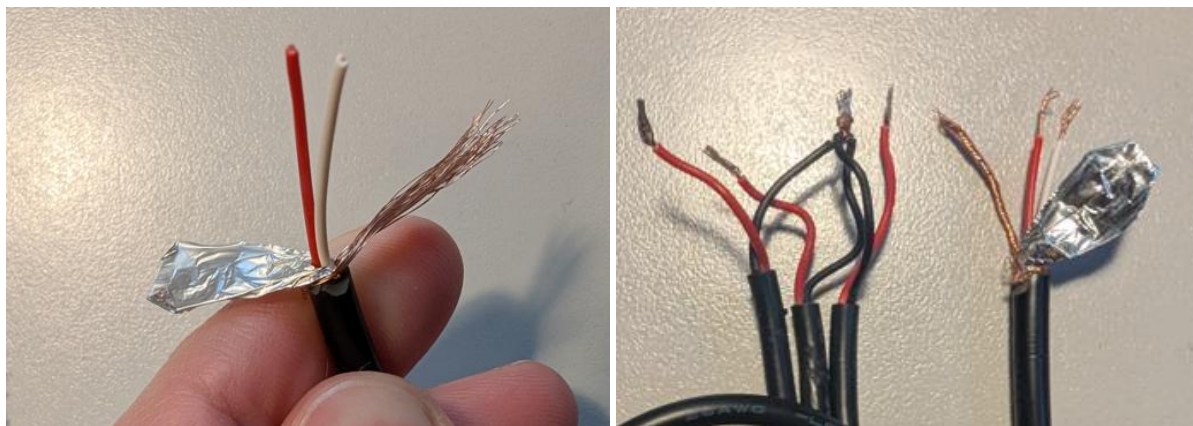
When looking for alternate parts on Digikey, we were unable to find exact replacements. Adafruit sold a TRRS terminal splitter, but it was significantly smaller than the one purchased from Amazon. There were also no direct equivalent to the mono cables, although there were similar 6ft long TRS cables that were ordered instead.

Once the parts arrived, a Switch Adapter was constructed and compared to the one made with Amazon parts. The first problem was the size of the TRRS splitter. The Adafruit one was significantly smaller, and requires a smaller screwdriver and the size of the openings on the terminal were much smaller. The Digikey cables were about a millimeter in diameter larger than the Amazon cables and had the ground wire as a wrapping of loose copper wires around the signal wires, wrapped in a metal foil.



DESIGN RATIONALE

When attempting to build the Switch Adapter, there were several problems that appeared. The first problem was with the screw terminals, using a screwdriver that was slightly undersized for the terminal size strips the flathead connector on the terminal block, the screwdriver used needs to be the largest possible one that can fit in the hole for the screw. The terminals themselves are also undersized, unable to fit the twisted copper ground wire from one of the wires, let alone three. The cables were also worse than the Amazon equivalent, being six feet long and TRS, instead of two feet and mono. When cutting down the cables and trimming them, the ground connector was a collection of copper wires wrapped around the signal wires which were themselves wrapped in a metal foil. These took significantly more work to prepare than the precut and prestripped Amazon cables, and resulted in an extremely bulky and fraying ground cable that did not fit in the terminal.



After this, the decision was made to continue with the Amazon components for easier makeability.

Limitations of the TRRS Trinkey

While the TRRS Trinkey microcontroller is designed for use with Assistive technology, it does have its limitations.

The largest limitation that was encountered during development was the memory available on the board. The board has 32 KB of RAM, and this gets used quickly. When developing the code, this memory limit was frequently hit. This memory limit was reached when there was still between 1-2 KB of memory remaining on the board, due to memory fragmentation where there is memory still left on the board, but not a single continuous section of memory able to store whatever the user is currently trying to store. The CircuitPython program does not have any defragmentation tools, so this issue prevented the settings file from being on a separate user editable file, as there was not room on the board to add a function to read it into memory. There are garbage cleanup tools available however, which remove unused variables from memory to free up as much space as possible.

As a consequence of the limited memory, various libraries and sub library functions such as `settings.toml`, are not supported on the board.

DESIGN RATIONALE

Another limitation faced was the hardware on the board. The board has six pins tied to the TRRS jack, one for each ring, one to for the tip switch, and one for the ring 1 switch. There is also a reset button on the board, an i2c plug, and a NeoPixel. Beyond this, there is no way to connect to the microcontroller. When the idea was raised of a separate mode button on the device instead of holding a switch input, there was no way to easily do this. It would have required either adding a separate i2c button board, or holding the reset button while plugging in the board.

Code Structure/Function

Time Monotonic Issue

A potential issue with the code is the use of time monotonic in the key press function. Time monotonic is used to track the duration of a keypress and determine when a key press reaches the mode change threshold. However, time monotonic is a 64-bit millisecond tick counter, and [begins to lose precision after 2²² milliseconds](#), or about 1.165 hours. At 2²³ milliseconds, it will change every 2 milliseconds, and so on. More precise measurements of time, such as `time.monotonic_ns()` are not available on the Trinkey TRRS due to the lack of [long integer support](#).

However, initial testing with leaving the device powered on for 16 hours it still took the same 4 seconds to change mode, which is roughly 2^{25.8} millisecond, which should take 8 milliseconds per tick instead of 1, or 32 seconds to change the mode. More testing is needed to determine how the device will function after long periods of time powered on. A blink test was suggested, where the Neopixel turns off and on at regular intervals, and can be informally monitored over time to track the change in the timing.

Adafruit TRRS Trinkey M0

```

analogio , array , board , builtins , busio , collections , digitalio ,
keypad , keypad.Keys , math , microcontroller , neopixel_write , nvm ,
os , random , storage , struct , supervisor , sys , time , touchio ,
usb_cdc , usb_hid , usb_midi
Frozen Modules: adafruit_hid

```

Figure 5: Supported CircuitPython Libraries for the Trinkey TRRS

One of the OFIs identifies was to add a settings file to allow the device to remember the mode after a reboot, and to make it easier for a clinician to change keystrokes or other settings without risking them breaking the code by changing the wrong section.

The requirements for a user adjustable setting file are:

- File must allow comments: Comments are needed to let the user know how to adjust the file, what they can and cannot change in the file
- File should be human readable: If the user is opening and changing the file, they should be able to read and understand the file
- File should be forgiving of simple formatting errors: Since the user will likely be editing the file in a text editor and not a proper IDE, an extra whitespace or newline shouldn't break the file

In CircuitPython, the two main ways of doing this is with a settings.toml file, or with a text file that you parse.

In CircuitPython, a settings.toml file is used to separate settings from the main code.py file to allow the main code to be shared without also sharing personal information, such as passwords or network settings. It supports comments, and is essentially an extension of the main code file used to declare variables. Unfortunately, although it supports the OS library, [the TRRS trinkey specifically does not support the use of settings.toml](#)

CircuitPython also allows for a txt file stored on the microcontroller to be read by the main program. While writing requires the board to be set to be read only to the computer, the text file can be read without changing settings on the board in a boot file.

If the data in the txt file is in JSON format, it can be easily read into the code using json.load. However, json does not support comments, is fairly rigid in its formatting, and is not very user parsable. [JSON is also not available on the Trinkey TRRS](#), a custom parser would need to be developed

YAML is a more user-friendly version of JSON that supports comments, but there is not a built in YAML parser the way that there is a JSON parser. A custom 'YAML lite' parser could be developed for the parts used in the keycode matrix.

Finally, a custom format can be developed for this specific use case, and a parser developed for it. This would take the most development time out of all the options, but provide the most flexibility and

DESIGN RATIONALE

customizability. An initial test showed that integers read from a text file can be used to send keystrokes, but so far, keycode objects cannot be stored in the text file and used to send keystrokes. Having the keystrokes stored as integers instead of human readable words would run counter to the goal of having the keystrokes in a separate file to make it easier for a maker to modify the keystrokes.

Options	Pros	Cons
Settings.toml	• Supports comments • Written in Python, allows declaring the list exactly as it would be written in the main code • User can swap files to get a different set of keystrokes • User doesn't have to open or make changes to the main file at all	• Not supported by the Trinkey TRRS
JSON	• Existing CircuitPython parser • User can swap files to get a different set of keystrokes • User doesn't have to open or make changes to the main file at all	• Built in circuitpython JSON parser is not supported by Trinkey TRRS • Would need to develop a stripped back parser • Comments are not supported by the standard • Standard is not very human readable
YAML	• Standard supports comments • Standard is more human readable • User can swap files to get a different set of keystrokes • User doesn't have to open or make changes to the main file at all	• Circuitpython does not have a built in YAML parser • Would need to develop a stripped back parser
Custom	• Would allow the creation of a standard that worked the best for the type of data we are storing • User can swap files to get a different set of keystrokes	• Would need to develop a custom parser • Can't just read in keystrokes from file, need a way to convert them from strings

DESIGN RATIONALE

	User doesn't have to open or make changes to the main file at all	
Leave keystroke matrix in	- Do not need to develop a custom parser - Do not need to develop a method to convert a string into a keycode object - Supports comments	User will have to open and make changes to the main file

Recommendation

While breaking out the keycode and other user settings into a separate file is a good idea, the limitations of the TRRS Trinkey eliminate the best options. A settings.toml file would be the easiest, most user friendly method, but does not work with the Trinkey. The same issue is present with using a JSON file, as the Trinkey does not support the CircuitPython JSON parser.

This leaves us with developing a limited parser that can read the data in the text file. At this point, the format can either be JSON, YAML, or a custom data format. Using JSON or YAML, code can be reused from an existing parser, but stripped back to work with the limitations of the hardware.

Developing a custom parser would take significant development time, and require optimization to fit in the hardware limitations of the microcontroller. It would also need error handling and edge case detection.

The recommendation for the Shrub Hub is to leave the keystroke matrix in the code.py file. The fact that none of the standard plug and play options would work, requiring a custom developed parser vastly increases the amount of development time required, for a small improvement to the user experience.

Implementation

When trying to add a series of dictionaries to the main code to allow strings to be associated with the corresponding code object, the memory limit of the board was reached. After removing the dictionaries, the available memory left was revealed to be 2528 bytes. However, just the dictionaries measured in another file only took 1312 bytes.

Some research indicated that this may be due to fragmented memory, where there is not a large enough continuous location in memory for the dictionary, [even though there is enough total room for them.](#)

Both declaring the dictionary at the beginning of the code, and frequent garbage collection were unable to make the dictionaries work. It also did not work when replacing the keycode.item calls with the integer values that they represent.

DESIGN RATIONALE

Another attempt was made to implement the settings closer to how they were handled in the Enabled Controller Mini CircuitPython version, with a section of the code dedicated to settings and separated from the rest of the code. In this section the individual button presses for each mode were broken out into their own variables, and all user modifiable settings were moved to this section. A declaration was added to then compile the keystroke matrix out of the component variables in the settings section. However, even this led to memory errors, as the addition of the extra variables for each keystroke entry caused memory issues.

Finally, an attempt was made to compile the code into a .mpy file, precompiling it to reduce the RAM usage and fragmentation using mpy-cross. The code was precompiled in powershell using `“./mpy-cross code_name”` and loaded onto the Shrub Hub as a library. The main code file was then changed to be `“import code_name”` (this does not work if the .mpy file is named code.mpy). The code used for this was a version of the code that read the keystrokes from a text settings file. However, even after being precompiled, the code still did not have enough memory. Another attempt was made, removing the settings file and keeping the settings in the main file while everything else was kept in the library. This was also unsuccessful.

At this point it was decided that the original single file code should be used for the initial release.

Persistent Mode

To allow the mode to persist across reboots, a method was found to write to a non-volatile memory (nvm) byte array. While Adafruit says that it shouldn't be possible to access non-volatile memory in circuitpython in non-express boards, the byte representing the mode was able to be stored to `microcontroller.nvm[0]`.

Testing

Test Methods

Test Switches

After connecting all switches to the commercial splitter cables, the device was plugged in and tested.

Test Results

Initially, there was an issue where one key was consistently pressed down. This was determined to be caused by the mono TS cable used in the switches being consistently shorted by the microphone splitter cable. As a temporary workaround, the switch was replaced with another switch that used a TRS jack that would not be shorted.

Once the cables were working, it was found that the software worked as described, sending u/d/s/l/r when the correct switch/chord was pressed.

User Testing

Two Shrub Hubs were sent to a pair of occupational therapists for testing. The initial pair sent were both programmed in Arduino but follow up instructions were sent to help with reprogramming the device in Circuit Python once that code had been developed. After the testers had the devices and used them with clients for about a month, a meeting was scheduled to get feedback on the devices

Feedback Questions

General Questions

- Inputs
 - Is 3 inputs enough? Too few? Too many?
 - I think 3 is good, most people will remember 3 macros, ties into a next/back/select type system for a lot of different systems
 - Were there times that you needed additional inputs?
 - I have used more than 3 inputs for macros but have never yet needed it as buttons instead of a number pad or macro board
- Chording
 - Were you able to use the chording?
 - Were you able to test with a user? Were they able to use chording?
 - Not yet
 - Would it be beneficial to add 2 more chorded inputs (1+3, 1+2+3)
- Enclosure
 - Do you have any thoughts or issues with the enclosure of the TRRS Trinkey?
 - It's good. I did try using it after putting lotion on my hands though and it was hard to remove, a few people might benefit from extra grip to pull it out, but mostly the user is not going to be the one connecting and disconnecting it.
 - A tactile label on the reset button would be helpful, visually it's not that clear where to press
 - Do you have any thoughts or issues with the enclosure of the mono cable splitter?

DESIGN RATIONALE

- I worry a little about the connection between the 2 getting bumped or pressed on and breaking the mono jack, it's not very stable . This could also happen easily when thrown in a bag.
 - I worry a little that some creators won't do a good job smoothing the ends of the cut zip ties, those can be really sharp if not cut carefully
 - It would be good to have the splitter mesh more with the trinke, maybe with clips to prevent disconnection
 - Did you have any issues telling what mode the device was in/seeing the light pipe when the device was plugged in?
 - No, but younger user had trouble holding for 4 seconds (kept rushing it)
 - Was the size of the combined trinke + splitter too big?
 - No. a right angle adapter could be used if needed?
- Getting the right input
 - Did you have any issues telling which switch input connected to which switch output?
 - No, your ring labels are clear and once the device is connected and programmed my memory moves to which switch color, not what number it is.
- Number of modes
 - Did you have any thoughts on the number of modes?
 - Did you find any of the modes unnecessary?
 - Never used 12345, too much chance for confusion
 - Were there any modes that you wanted but were not in the device?
 - mouseclicks
- Mouse clicks
 - Would mouse clicks be something you would want added?
 - Definitely, this is the most common use when pairing with something like a headmouse, and is effective to replace fine motor or finger control functions. Good for temporary disabilities
 - Would mouse movement be something you want added
 - Directional movement (pressing a button to move in the different directions)
 - I think that the other MMC joysticks already accomplish this
 - Morse mouse (Morse code to move the mouse around)
- Media control
 - Would media control (play/pause, skip, rewind, etc) be worth adding?
 - Would that be worth adding another mode for?
 - Yes, additional modes aren't a burden for the user and most people wouldn't reprogram them.
- Morse Code
 - Would morse code typing be worth adding?
 - I have only worked with one morse user and know of one more, both use a darcy/tandem master - I don't think new users are adopting the technology very much.

Arduino Version

- Were there any issues changing the mode in Arduino mode?

Programming Circuit Python



© 2026 by Neil Squire.

This work is licensed under the CC BY SA 4.0 License: <http://creativecommons.org/licenses/by-sa/4.0>Files available at <https://github.com/makersmakingchange/Shrub-Hub>

DESIGN RATIONALE

- How would you describe the experience programming the hub with CircuitPython
 - Works well, it would be nice if there was a reminder of color within the matrix
- Were there any parts of programming the device that you found confusing or had questions about?
 - Double clicking button brings up a different boot menu
 - How to get out of boot menu/stop explorer opening folder

Circuit Python Version

- Were there any notable differences between this version and the Arduino version. If so, did they make the device better or worse
- Did you notice any issues with changing mode being slower after the device was plugged in for 2 hours? 4 hours?
 - Sorry, didn't test it for that long yet 😞
- Were you able to change the keystrokes in the code?
 - Where you able to create a compound keystroke (like Win + H)

Feedback Summary

- Three buttons is enough, a one button mode would be a nice addition
- Didn't have an opportunity to test chording
- Trinkey was hard to remove from computer sometimes, enclosure is hard to grip
- Reset button should be more tactile/stand out more. Sometimes try to press the light pipe accidentally since it sticks out more.
- Connection between splitter and hub is delicate, and worry about breaking it when throwing it into a bag
 - Maybe add some clips to make the splitter attach better
- Cable ties on splitter could be sharp if a maker doesn't carefully trim them down
- Some younger users had difficulty holding it for 4 seconds and kept releasing too soon. Adding a blinking light when 4 seconds is up would help
- Right angle adapter might be useful but not necessary
- Ring labels on splitter are clear
- Didn't use the 12345 mode, too much chance for confusion
- Really want the addition of mouse clicks
- Would want the addition of media control
- Don't think that there's much demand for morse typing
- When in circuitpython mode, device was easy to reprogram
- Would be nice to have a comment/reminder of the mode colour in each row of the keystroke matrix
- Accidentally broke the code when trying to make changes, need to add a more in depth troubleshooting to the user guide about recognizing when the code is broken, since it is interpreted instead of compiled

Shrub Hub

DESIGN RATIONALE



OFls

- Add gripable spots to the side of the enclosure
- Design a better enclosure for the splitter that does not rely on zip ties
- Create a stronger connection between the splitter and the hub
- Add mouse click mode
- Add media player mode
- Remove 12345 mode
- Improve troubleshooting section of user guide
- Better comments in the code
- Make it so the light flashes once 4 seconds holding the button is reached
- Make the reset button more tactile
- Save mode between reboots

Shrub Hub

DESIGN RATIONALE



Design Output

Naming

When choosing a name for the device, an effort was made to stay within the existing forest/tree theme for MMC joysticks and digital interfaces. A variety of names were brainstormed, such as Glade, Grove, Weald, and more names were suggested by the tester, such as Shrub, Arbor, Orchard, Leaf, and Twig.

Weald was eliminated first, as it was an obscure word that nobody knew. Grove was eliminated due to being the name of a connector by Seeed Studio, while Glade was eliminated to avoid overlap with the air freshener brand. From the remaining list of names, the name Shrub was chosen due to the connotations of being a small tree/plant.

The name of the add-on device that splits the TRRS jack into three mono cables was also something that needed to be determined. It was initially named the Switch Splitter, but the issue was raised that it sounded like a device that could be used to split one switch to instead control several devices. The following list of replacement names were brainstormed:

- Switch Adapter
- Splitzio
- Switch Breakout
- TripSplitz
- Switch Merger
- 3-in-1 Switcher
- 3-to-1 Switch
- 3-2-1 Switch
- 3-2-1/3-to-1 Merger

The decision was made to go with 3-to-1 Switch Adapter for this device.

Shrub Hub

DESIGN RATIONALE

Detailed Design

Overview

The device is comprised of two main parts, a TRRS Trinkey microcontroller in an enclosure, and a TRRS breakout connector that allows three assistive switches to be connected to the device. When connected to the computer, the device will send keystrokes when the connected buttons are pressed. It has five different modes with different sets of keystrokes, and the user can change between them by holding down a connected switch for four or more seconds. The different modes allow users to send screen navigation keystrokes such as space, tab, and enter, send the first five functions keys(F1-F5), perform mouse clicks, send the first five unused function keys(F13-F17) for custom hotkeys, and perform media control such as play/pause, volume up/down, and skip.



Figure 6: Finished Shrub Hub and Switch Adapter

Shrub Hub

DESIGN RATIONALE

Enclosure

Objectives:

1. Protects the internal board
2. Provides grip to more easily insert/remove the device.
3. Improves the aesthetics of the device
4. Provides a connection point for the Switch Adapter
5. Provides a place for labelling and branding

Requirements:

1. Hold the PCB securely
2. Show the mode LED
3. Provide access to the reset button

Criteria

1. As narrow as possible.
2. Minimal print time.
3. No or minimal tools required for assembly.
4. Aesthetic

Shrub Hub

DESIGN RATIONALE

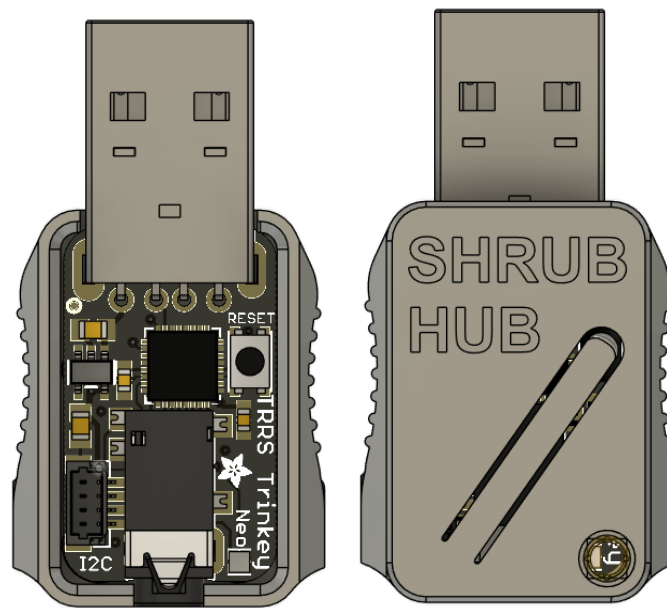


Figure 7: Shrub Hub with and without top of enclosure

The enclosure for the microcontroller is a two-part enclosure that uses a snap fit to connect the lid to the base. The microcontroller is held in place vertically by the fit of the two halves around the USB and TRRS connectors, and four pillars on the bottom that space the microcontroller from the bottom, and horizontally by the general shape of the base of the enclosure. There is ~0.23 mm of clearance between the walls of the enclosure and the sides of the PCB.

The enclosure keeps the flexible arm to activate the reset button from the original Adafruit enclosure, but the pillar that presses the reset button has deliberately been made too tall so the arm flexes out beyond the plane of the lid of the enclosure to provide a tactile locating feature to make it easier to press the reset button when the enclosure is not visible, such as when the device is plugged into the back of a computer.

A light pipe has been added to the lid of the enclosure, located over the microcontroller's NeoPixel to allow the user to view the visual feedback from the light. The light pipe is held in place using a press fit.

Finally, the exterior of the enclosure has two concave detents with vertical ribs to make it easier to grip the enclosure while removing it from the computer, and two integrated snap fit pockets to connect to the Switch Adapter.

Shrub Hub

DESIGN RATIONALE



Switch Adapter

The TRRS Trinkey has a TRRS input. To use standard assistive switches with mono plugs, an adapter is required. There is a combination of commercially available adapters that can work, but they are expensive, only specific varieties will have the correct wiring, and the overall connection is not compatible with all assistive switches.

Using a custom adapter ensures that the device is compatible with all assistive switches. The adapter consists of an off-the-shelf TRRS terminal block, three mono cables with female jacks, and a 3D printed enclosure.

Objective

1. Provides an easy way to connect multiple mono switch connections to the TRRS input on the Shrub Hub

Requirements

1. Must be able to connect three 3.5 mm mono plugs in the correct wiring scheme to a 3.5 mm male TRRS plug.
2. Must be able to be assembled without soldering.
3. Mono switch connections must be labelled.

Criteria

1. Durable, reliable connection to Shrub Hub
2. Minimal size
3. Durability
4. Reliability

The Switch Adapter consists of a 3D printed enclosure, a screw terminal TRRS jack, and three female mono cables

The mono cables are attached to the screw terminals of the TRRS jack and allow the TRRS connection to be split out to the three mono cables with a common ground. The enclosure protects these connections and allows for the Switch Adapter to be more securely connected to the microcontroller enclosure.

The enclosure consists of two parts: a base that connects to the microcontroller enclosure and grips the mono cables; and a lid that attaches the enclosure to the TRRS jack. The two halves of the enclosure connect with a snap fit.

The base of the Switch Adapter has three primary features. The first feature is the two long arms that extend from the body of the splitter. When the TRRS jack is connected to the microcontroller, these arms clip into the detents in the body of the microcontroller enclosure and help to prevent the TRRS jack from being unplugged or coming loose. These attachment arms also feature two side arms that can be gripped to flex the enclosure arms to the side to detach them from the microcontroller enclosure. The



DESIGN RATIONALE

second main feature of the base are the flexible arms that grip the three mono cables. Designed for use with 3mm mono cables, these arms flex out of the way, with the vertical ridges gripping the cable, preventing it from moving forwards or backwards. Finally, the base contains slots for the snap fit of the lid to attach.

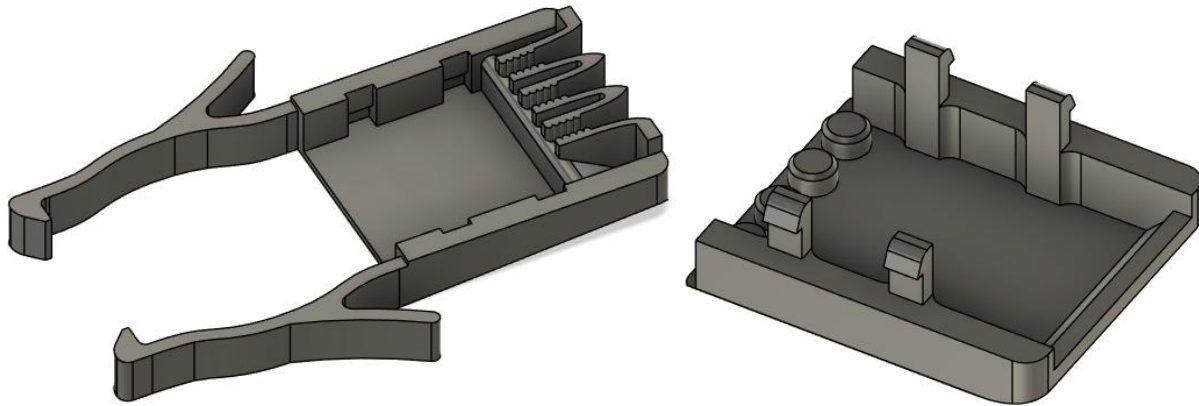


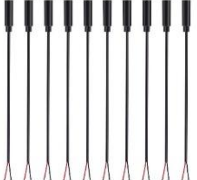
Figure 8: Top and Bottom of Switch Adapter Enclosure

The lid of the enclosure contains two main features, the snap fit arms that attach it to the base, and the four posts that help retain the enclosure on the splitter. The enclosure is assembled around the TRRS splitter, and the four posts are inserted into the holes of the screw terminal, preventing the enclosure from moving forwards or backwards relative to the TRRS jack. The enclosure is prevented from moving side to side by a combination of the posts and the fit of the enclosure around the jack and prevented from moving up and down by the fit of the enclosure around the jack.

DESIGN RATIONALE

Electrical Components

After the issue with the mono cable switches shorting on the microphone splitter, an alternate splitter cable was created using a TRRS terminal jack and three pre stripped female mono cables. This option requires slightly more assembly but still does not require soldering. It is cheaper to build than the commercial splitter version and gets even cheaper when built in bulk.

Part	Image	Price	Link
Single Build			
TRRS Terminal		\$6.58/2 pack \$3.28 Each	Link
Female Mono Cable		\$10.99/4 pack \$2.75 Each	Link
Bulk Build			
TRRS Terminal		\$14.93/8 pack \$1.86 Each	Link
Female Mono Cable		\$20.22/10 pack \$2.02 Each	Link

The final wiring of the TRRS adaptor is as follows, where the bolded section is the terminal on the TRRS plug, and the rest is the wire that is connected to it:

- **L**: Common ground for all three switches
- **R**: The signal cable for switch 2
- **V**: The signal cable for switch 1
- **GND**: The signal cable for switch 3

This configuration allows for a single switch to be plugged in to the Shrub Hub instead of the adapter and function as either Switch 6 or 7, depending on the cable type used on the single switch.

Shrub Hub

DESIGN RATIONALE

Code Structure / Function



Overview

The code for the Shrub Hub is written in CircuitPython and can be easily loaded onto the device with minimal instructions, and settings easily changed. The code detects which combination of the three switches have been pressed and determines which keystrokes should be sent based on the combination of switches pressed.

Platform

The Shrub Hub was programmed using CircuitPython on the Mu IDE, as opposed to most MMC devices, which use ArduinoC. The decision was made to program the device in CircuitPython for several reasons. One of the largest reasons was to allow for the user to be able to change settings in the firmware easier, as the code is stored on the device and can be easily modified without needing to install anything. Programming in CircuitPython also allows the MMC team to build institutional knowledge on using CircuitPython and its limitations.

Operating Modes

The device currently has five different operating modes, based off the original operating modes of the Adafruit version, and feedback from the testers. The mode can be changed by holding down any connected switch for more than the mode change time, which by default is four seconds. Each mode by default has five different keystrokes, three of which are activated when pressing an individual switch, and two that are activated by pressing two switches at the same time. The default modes that the device is programmed with are as follows:

- Mode 1: Enter/Space/Tab/Backspace/Escape
- Mode 2: F1/F2/F3/F4/F5
- Mode 3: Left Click/Right Click/Middle Click/Left Click/Right Click
- Mode 4: F13/F14/F15/F16/F17
- Mode 5: Play/Volume Up/Volume Down/Next Track/Previous Track

The keystrokes of mode one were chosen based on the research done on existing commercial hubs, as they were commonly found on most commercial hubs and can be used for basic computer navigation without needing any settings changed on the host device. Mode 2 keystrokes were taken from the MMC Enabled Controller. Mode 3 and 5 were added after feedback from testers, and mode 4 was added on the request of a tester who wanted unused keys that could be used with the AutoHotkey program to trigger different actions and programs.

After full release of the device one of the goals is to get feedback from users and clinicians about the range of keystrokes on the Shrub Hub and ensure that they meet the needs of the users.

Input Handling

When a switch is pressed, the NeoPixel is turned off, and the device waits until the switch is released. Once the switch has been released, if it was held for less than four seconds the device turns the



DESIGN RATIONALE

NeoPixel back on and sends the appropriate output to the connected computer. If the switch was held for over four seconds, the mode changes, the colour of the NeoPixel changes, and no output is sent to the connected computer. **It is important to note that the output is sent to the connected device when the switch is released, not when it is pressed.**

The device checks for switch presses at 100Hz, or 100 times a second.

Shrub Hub

DESIGN RATIONALE



Operation

Once plugged in, the Shrub Hub reads the last stored mode from non-volatile memory, then enters the main loop. The main loop of the Shrub Hub firmware constantly checks to see if a button has been pressed. If it has, the program turns off the LED and checks to see which buttons have been held down and checks to see when they are released. Once the buttons have been held for longer than the mode change time, the LED comes back on. Once the buttons are released, if they were held for longer than the mode change time, the mode will change, the LED colour will update, the new mode will be logged to the non-volatile memory, and no keystroke will be sent. If the buttons were held for less than the mode change time, the buttons pressed will be sent to the keystroke compiler.

The keystroke compiler takes the buttons pressed and the mode and looks up the corresponding entry in the keycode matrix. The entry contains a number corresponding to the type of keystroke (keystroke, mouse click, or media control), as well as a series of keystrokes. From here, it will use the identifier number to determine which library to use and sequentially press and hold all the keys listed in the matrix entry. This is largely irrelevant for mouse and media control, which only use one key at a time, but allows combination keystrokes such as CTRL+C or CTRL+SHIFT+T. After all keys are pressed, the keystroke is sent, all keys are released, and the program returns to the main loop.

Currently, switches 1, 2, and 3 all have their own corresponding outputs, while the combination Switch 1+2 and switch 2+3 also have their own distinct outputs. Switches 1+3 and 1+2+3 send the same output as switch 1, as it allows for a single switch to be plugged directly into the TRRS jack on the Shrub Hub without the use of the Switch Adaptor and still function.

DESIGN RATIONALE

Validation and Verification Plan

To verify and validate the Shrub Hub, we review the goals, requirements and constraints that were established at the beginning of the project and compare them with the finished device. At the beginning of the project, we established the following:

Goals

G01	Create a low-cost switch interface
G02	Create an easy to build switch interface

Functional Requirements

F01	Device must accept up to two 3.5 mm switch inputs
F02	Device must output HID keystrokes to the connected device

Constraints

C01	Device cannot use a custom PCB
C02	Device cannot require soldering

Now that the device is further along in development, we can compare it with the goals, requirements and constraints.

Goals

- G01: The Shrub Hub has a unit cost of \$27.19 CAD and a total cost of \$33.01 CAD, compared to the Rocket Switch, which has a unit cost of \$21 and a total cost of \$45. This makes the Shrub Hub cheaper for a single unit build, and similar for a bulk build to the Rocket Switch, which is the MMC budget switch interface. This is a pass.
- G02: The Shrub Hub can be assembled with no soldering and only requires a screwdriver to assemble the Switch Adapter. This is a pass.

Functional Requirements

- F01: The Shrub Hub can connect to three 3.5 mm switch inputs. This is a pass
- F02: The Shrub Hub sends three sets of five HID keystrokes, as well as a set of five mouse clicks and five media control commands. This is a pass

Constraints

- C01: The device does not require a custom PCB. This is a pass
- C02: The device does not require soldering to assemble. This is a pass.

DESIGN RATIONALE

After reviewing the goals, requirements, and constraints set out in the beginning of the project, the Shrub Hub meets all of them

This section defines the verification and validation (V&V) activities required to confirm that the device meets its design requirements, user and stakeholder needs, and applicable safety and performance expectations.

- Verification ensures the design was built correctly according to specifications.
- Validation ensures the right design was built, meeting user needs in real-world use.

Verification

Verification Objective

Confirm that all design requirements have been met through objective evidence such as inspection, analysis, or testing.

Verification Methods

Common verification methods include:

- Inspection: Visual or measurement confirmation (e.g., enclosure dimensions, solder joints).
- Analysis: Analytical confirmation (e.g., thermal calculations, firmware memory usage).
- Test: Functional testing under controlled conditions (e.g., switch response time).
- Demonstration: Simple operational proof (e.g., assembly demonstration).

Shrub Hub

DESIGN RATIONALE

Verification Matrix

Requirement ID	Requirement	Verification ID	Verification Method	Acceptance Criteria	Result / Status (Pass/fail, comment?)	Evidence Reference (linking to results/data that showed we passed the requirement)
N/A	Shrub Hub enclosure must be printable on a high-end printer (Bambu P1S, Prusa Mk4)	V01a	Demonstration	The enclosure can be printed on a high end printer with text visible and all features functional	Pass	
N/A	Shrub Hub enclosure must be printable on an older printer (Ender 3)	V01b	Demonstration	The enclosure can be printed on an older printer with text visible and all features functional	Pass	
N/A	Switch Adapter enclosure must be printable on a high end printer (Bambu P1S, Prusa Mk4)	V02a	Demonstration	The enclosure can be printed on a high end printer with text visible and all features functional	Pass	
N/A	Switch Adapter enclosure must be printable on an older printer (Ender 3)	V02b	Demonstration	The enclosure can be printed on an older printer with text visible and all features functional	Pass	
N/A	Reset button must not get stuck in use when printed on a high end printer	V03a	Demonstration	When pressing the reset button on the assembled device, the lever arm	Pass	



DESIGN RATIONALE

	(Bambu P1S, Prusa Mk4)			should spring back and not remain pressed		
N/A	Reset button must not get stuck when printed on an older printer (Ender 3)	V03b	Demonstration	When pressing the reset button on the assembled device, the lever arm should spring back and not remain pressed	Pass	
N/A	The reset button must last 1000 presses	V04	Test	The reset button should be able to be pressed a thousand times and still work without getting stuck.	Pass	
N/A	The reset button arm should still work after 1 month	V05	Demonstration	The reset arm should not creep enough to stop working after a month	Pass	
N/A	The device must be able to be cleaned with a damp cloth.	V06	Demonstration	The Shrub Hub can be cleaned with a damp cloth.	Pass	
N/A	The light pipe should be installable with less than TBR and more than TBR grams of force	V07	Test	The user should be able to install the light pipe with less than TBR and more than TBR grams of force		
N/A	The Shrub Hub enclosure must last 25 snap fit cycles	V08	Test	The enclosure should be able to be installed and removed 25 times without breaking	Pass	
N/A	The Switch Adapter enclosure must last 50 snap fit cycles	V09	Test	The enclosure should be able to be installed and removed 50 times without breaking	Pass	
N/A	The Shrub Hub must withstand a 100 cm drop onto concrete	V10	Test	The Shrub Hub should be dropped from 100 cm onto	Pass	

Shrub Hub

DESIGN RATIONALE

				concrete and inspected for damage		
N/A	The Shrub Hub with Switch Adapter must withstand a 100 cm drop onto concrete	V11	Test	The Shrub Hub with Switch Adapter should be dropped from 100 cm onto concrete and inspected for damage	Pass	
	With a single mono switch connected, the Shrub Hub should send the first input of the active mode	V12a	Demonstration	Connect a mono switch to the Shrub Hub, press it and monitor the output.	Pass	
	With a single TRS switch connected, the Shrub Hub should send the first input of the active mode	V12b	Demonstration	Connect a TRS switch to the Shrub Hub, press it and monitor the output.	Pass	
	Holding any switch for more than the mode change time should change the mode	V13	Demonstration	Connect a switch to the Shrub Hub, press it for more than the mode change time and verify that no output is sent and the mode changes.	Pass	
	A switch connected to the switch 1 port of the Switch adapter should send the first input of the active mode	V14a	Demonstration	Connect a switch to the Switch Adapter port 1, press it and monitor the output.	Pass	
	A switch connected to the switch 2 port of the Switch	V14b	Demonstration	Connect a switch to the Switch Adapter port 2,	Pass	

DESIGN RATIONALE

	adapter should send the second input of the active mode			press it and monitor the output.		
	A switch connected to the switch 3 port of the Switch adapter should send the third input of the active mode	V14c	Demonstration	Connect a switch to the Switch Adapter port 3, press it and monitor the output.	Pass	
	A switch connected to the switch 1 port of the Switch adapter pressed at the same time as a switch connected to the switch 2 port should send the fourth input of the active mode	V14d	Demonstration	Connect a switch to the Switch Adapter port 1 and one to port 2, press them and monitor the output.	Pass	
	A switch connected to the switch 3 port of the Switch adapter pressed at the same time as a switch connected to the switch 2 port should send the fifth input of the active mode	V14e	Demonstration	Connect a switch to the Switch Adapter port 3 and one to port 2, press them and monitor the output.	Pass	
	A switch connected to the switch 1 port of the Switch adapter pressed at	V14f	Demonstration	Connect a switch to the Switch Adapter port 1 and one to port 3, press them and monitor the output.	Pass	

DESIGN RATIONALE

	the same time as a switch connected to the switch 3 port should send the first input of the active mode					
	When switches connected to all three switch ports are pressed at the same time, the device should send the first input of the active mode	V14g	Demonstration	Connect a switch to each Switch Adapter port, press them all at the same time and monitor the output.	Pass	
	All outputs should match the outputs described in the User Guide output table	V15	Demonstration	Test every mode and switch combination, monitor the outputs, and compare them to the ones on the table.	Pass	
	The colour of the mode indicator should match the colours listed on the output table for the different modes.	V16	Demonstration	Cycle through the modes and compare the colours to the ones on the table	Pass	
N/A	The Shrub Hub should not send outputs when plugging in a switch	V17a	Demonstration	The Shrub Hub should be connected to a computer and a switch plugged in while outputs are monitored	Pass	
N/A	The Shrub Hub should not send outputs when	V17b	Demonstration	The Shrub Hub should be connected to a computer and the Switch Adapter	Pass	

DESIGN RATIONALE

	plugging in the Switch Adapter			plugged in while outputs are monitored		
N/A	The Shrub Hub should not send outputs when plugging a switch into the Switch Adapter	V17c	Demonstration	The Shrub Hub with Switch Adapter should be connected to a computer and a switch plugged in while outputs are monitored	Pass	
N/A	The cables of the Switch Adapter should not come loose if a switch is improperly unplugged.	V18	Demonstration	A switch should be connected to the Switch Adapter and unplugged with one hand pulling on the switch plug and the other holding the enclosure of the switch adapter.	Pass	
N/A	The cables of the Switch Adapter should not come loose if a switch is improperly unplugged after being installed for a month.	V19	Test	A switch should be connected to the Switch Adapter and unplugged with one hand pulling on the switch plug and the other holding the enclosure of the switch adapter. This test should be performed on a Switch adapter that has been assembled for a month.	Pass	
N/A	The Switch Adapter should not come loose if a switch is improperly unplugged.	V20	Demonstration	A switch should be connected to the Switch Adapter while plugged into the Shrub Hub and unplugged with one hand pulling on the switch plug	Pass	

Shrub Hub

DESIGN RATIONALE

				and the other holding the enclosure of the Shrub Hub		
N/A	The timing of the device should not drift more than 10 percent in a week.	V21	Test	A test program should be loaded onto the microcontroller that blinks the microcontroller every 10 seconds. This program should be left to run for a week and the timing of the blinks measured.	Pass	
N/A	The device should work equally as well on Windows, Linux, Android, iOS and MacOS	V22	Demonstration	The device should be used on each of the operating systems to be tested and send the required outputs.	Tentative pass	
N/A	A user should be able to plug a USB drive into the port next to the device.	V23	Demonstration	The Shrub hub with Switch Adapter should be plugged into a USB port and several USB devices plugged into the port next to it.	Tentative Pass	Can plug the USB flash drives next to the shrub hub, but only thin flash drives can be plugged in next to it when the switch adapter is connected
N/A	The Shrub Hub Enclosure should work with all TRRS Trinkey Boards	V24	Demonstration	The first 10 Shrub Hubs built by MMC should be inspected for fit of the enclosure and that the lid fully snaps into place.		

DESIGN RATIONALE



N/A	Shrub Hub enclosure should fit the TRRS Trinkey PCB even when printed on an older printer (Ender 3)	V25	Demonstration	The TRRS Trinkey should fit into the Shrub Hub enclosure when it is printed on an older printer	Pass	
	Can the Switch Adapter be constructed with different cables	V26	Demonstration	The Switch Adapter should be attempted to be constructed using a variety of cables.	Fail	Larger cables do not work with the cable retention arms

Shrub Hub

DESIGN RATIONALE

Verification Summary



Shrub Hub

DESIGN RATIONALE

Validation



Validation Objective

Confirm that the final design meets user and stakeholder needs in its intended context of use.

Validation Questions

The following list of questions was used for brainstorming what to ask the users on the Microsoft Form.

- External Build Validation
 - Can a user without a technical background build the device following the Maker Guide?
 - Can a user without a technical background modify the outputs?
 - Can a user with a technical background modify the outputs?
 -
- Shipping Validation
 - Does the finished device fit in letter mail?
 - Does the finished device fit in oversized letter mail?
 - Does the device require padding to ship safely?
- User Validation
 - Can a user set up the device following the user guide
 -

User Validation

Shrub Hub

DESIGN RATIONALE

Validation Matrix

Validation ID	Validation Objective	Participant(s)	Procedure	Acceptance Criteria	Result / Comment
End-user Validation					
Val01	Ensure that a user can set up the device using the User Guide		Provide prototype for participants to test.	Testers verify that they can follow the user guide and test the device	
Val02	Ensure that a primary user can send single switch outputs		Provide prototype for participants to test.	Primary user uses a connected assistive switch to send an output	
Val03	Ensure that a primary user can send two switch outputs		Provide prototype for participants to test.	Primary user uses two assistive switches to send a chorded output	
Val04	Ensure that a primary user can change the mode of the device		Provide prototype for participants to test.	User holds a button for more than the mode change time and changes the mode of the device.	
Val05	Determine appropriate default mode change time		Provide prototype for participants to test.	Testing participants provide appropriate default mode change times.	
Val06	Determine if users would be better served by a different default set of outputs		Provide prototype for participants to test.	Testing participants provide appropriate default outputs.	
Val07	Ensure the device is an acceptable size.		Provide prototype for participants to test.	Testers verify the Shrub Hub is an acceptable size with	



Shrub Hub

DESIGN RATIONALE

				and without the Switch Adapter.	
Val08	Ensure the device is an acceptable weight.		Provide prototype for participants to test.	Testers verify the Shrub Hub is an acceptable weight with and without the Switch Adapter.	
Val09	Ensure the User Guide is not missing information.		Provide participants with the User Guide to read through and reference when testing the Shrub Hub.	Secondary users provide feedback on what information should be added to the User Guide.	
Val10	Ensure the User Guide is written clearly.		Provide participants with the User Guide to read through and reference when testing the Shrub Hub.	Secondary users provide feedback on how to make the User Guide clearer.	
Val11	Secondary user is able to change the outputs of the device		Provide participants with the User Guide and Shrub Hub.	Secondary user is able to modify the code of the Shrub Hub to change the outputs	
Volunteer Maker Validation					
Val12	Ensure a maker without a technical background can build the device.		Have volunteer maker build the device	Volunteer maker is able to build the Shrub Hub following the maker guide	
Val13	Ensure the device can fit through a Canada Post	Brad	Have volunteer maker mail the device	Maker is able to fit the packaged device through an oversized Canada Post mail slot	Pass

DESIGN RATIONALE



	oversized letter mail slot				
Val14	Ensure that the device does not get damaged in the mail.		Have volunteer maker mail the device	Maker is able to mail the device and it arrives without damage	

Shrub Hub

DESIGN RATIONALE



Shrub Hub

DESIGN RATIONALE

Validation Summary



Shrub Hub

DESIGN RATIONALE

Design Transfer



Media

MMC Library Main Photo

- Using photo of Hub w/out Adapter✓
-

MMC Library Listing Photos

- Standalone Hub Photo ✓
- Hub w/ Adapter Photo ✓
- Attached to Laptop Photo ✓
- Attached to Phone Photo ✓
- With USB-A extender? Photo✓
- LipSync -style connection diagram ✓ ✓
- Output functionality
 - o (table) ✓
- Line-art icon ✓ ✓

Licensing

Hardware: CERN-OHL-W v2

Software: GNU General Public License v3.0 (GPL-3.0).

Documentation: CC BY-SA 4.0



© 2026 by Neil Squire.

This work is licensed under the CC BY SA 4.0 License: <http://creativecommons.org/licenses/by-sa/4.0>

Files available at <https://github.com/makersmakingchange/Shrub-Hub>

Shrub Hub

DESIGN RATIONALE



Attribution

The device was designed by Neil Squire Society.

The Shrub Hub is built using the following commercially available open-source hardware components:

- [Adafruit TRRS Trinkey - USB Key for Assistive Technology](#). The PCB design is available under a CC-BY-SA 3.0 license at <https://github.com/adafruit/Adafruit-TRRS-Trinkey-PCB>. It is certified as open hardware with the Open Source Hardware Association (OSHW) as [US002639](#).

The Shrub Hub is programmed in [CircuitPython](#) using the [Mu Python Editor](#)

Contributors

- Brad Wellington. Shrub Hub Enclosure, firmware, documentation. Neil Squire Society / Makers Making Change.
- Bill Binko. TRRS Trinkey PCB design and Learning Guide. [ATMakers.org](#).
- Chris Young. TRRS Trinkey PCB Design. [ATMakers.org](#).
- Eric Chau. Testing. Neil Squire Society / Solutions.
- Jake McIvor. Documentation, testing. Neil Squire Society / Makers Making Change.
- Limor Fried. TRRS Trinkey PCB Design. [Adafruit](#).
- Liz Clark. TRRS Trinkey Learning Guide. [Adafruit](#).
- Jody Dickerson. Testing. Neil Squire Society / Solutions.
- Josie Versloot. Testing. Neil Squire Society / Makers Making Change.
- Stephan Dobri. Testing. Neil Squire Society / Makers Making Change.

Delivery / Shipping Information

	Value	Notes
Total Shipping Mass	TBC grams	
Total Shipping Envelope	TBC mm x TBC mm x TBC mm	

Functionality

- Investigate mouse cursor control
- Investigate mouse scroll wheel modes
- Investigate utility of mode changes being controllable by the primary user
 - Changing to something like the Ivy mode change where it changes mode if it is plugged in while a certain button is held would allow the code to be changed to allow holding inputs/activating inputs on button press

Enclosure

- Height of USB connector varies from board to board, causing minor fit issues between boards
- Add feature to provide for mounting?

Electrical Components

- Better documentation on how to determine which input goes to which ring on the TRRS/which switch input
- Examine options for a better method of labeling the switch inputs than the rings of silver sharpie.

Code Structure / Function

- The main improvement that needs to be made is the addition of a settings mode like in the Rocket Switch.
- The mode could be saved so it does not need to be reselected after unplugging the device.
- Detect when only the stereo splitter is in use and disable the third switch input so it is not constantly pressing
 - This could be extended to disable any input that is held for too long and maybe send an error?
 - Detect when a cable is plugged/unplugged to allow it to scan for shorted inputs and
- Detect when jack is connected?
- How do we make it easier for a user to reduce the number of operating modes?
- How do we make it easier for operating mode changes to be disabled?
- How do we make it easier to enable / disable chording entries?
- Adding a gaming mode
- Adding a way to allow for sending keystrokes on a button press/allowing for keystrokes to be held

Shrub Hub

DESIGN RATIONALE



Appendix

Appendix A – Verification Test

f